그래픽스&엔진 포트폴리오



이 윤 상

Game Client · Real-time Path Tracing · AI



GitHub github.com/yunsang1ee

Youtube youtube.com/@profit_award

이 표시는 깃허브 링크입니다

저를 소개합니다

게임 클라이언트와 실시간 렌더링을 중심으로 시스템을 설계하고 구현합니다



이윤상

게임공학과 주전공 · 인공지능학과 부전공

- Real-time raytracing(pathtracing)
- Rendering engine
- GPU/low-level optimization
- AI for Graphics

이 모든 것을 연결해 결과적으로 엔진 성능 향상과 photorealistic graphics를 구현하는 일을 지향합니다

Rendering

DXR, Path Tracing, ReSTIR PT, DLSS 4.5

Client Architecture

Scene, ComponentStorage, DenseList, RenderPass

Game Systems

WinAPI, OpenGL, DirectX 12, Resource Pipeline

AI Systems

FastAPI, MCP, OpenSearch, Graph RAG

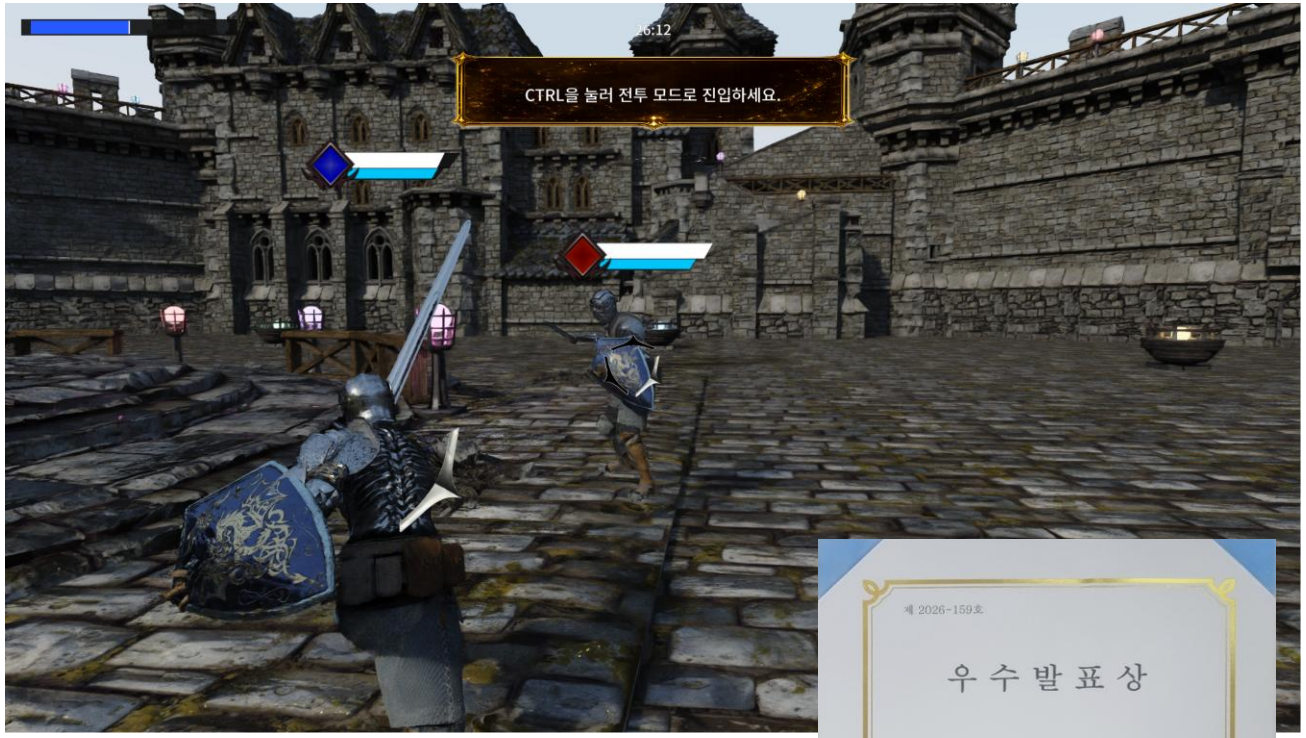
PROJECTS

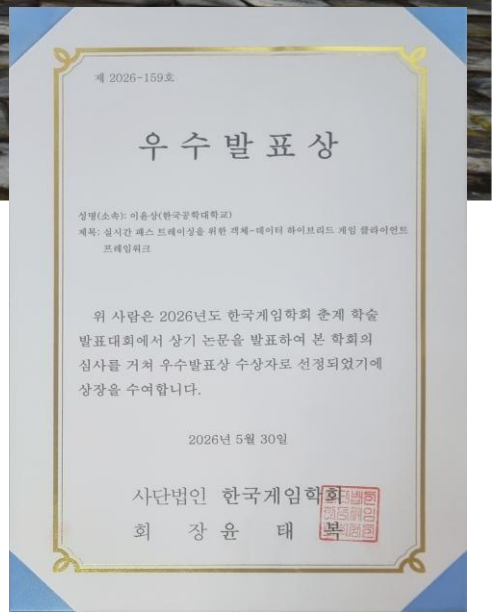
메인 프로젝트인 EisenValor는 깊게 보여주고, 이후 프로젝트는 기술을 축으로 설명하겠습니다.

01	**EisenValor**	ReSTIR PT + DLSS 4.5 기반 Real-time Path Tracing 게임 클라이언트
02	**RayTracingSimulator**	OpenGL Compute Shader 기반 GPU Path Tracing
03	**MetalSlug Clone**	WinAPI 기반 2D 액션 게임과 자체 클라이언트 구조
04	**AgenticGraphRAG**	FastAPI, FastMCP, OpenSearch 기반 Graph RAG 검색 시스템
05	**RigidPhysGS**	3DGS 시각 특징 모델과 물성 prior 연구

프로젝트 개요

프로젝트 및 장르	EisenValor (3D MO 전략 액션)
기간	2025.06.30 ~ 2026.06.22
인원	프레임워크&그래픽스 1명, 클라이언트 1명, 서버 1명
분야 및 담당	엔진 프로그래밍 클라이언트 프레임워크, DXR 렌더 파이프라인, ReSTIR PT, DLSS 4.5 SR/RR 적용
사용 툴	Visual Studio2022, Unity, PIX, recast, 자체 뷰어, Claude Opus4.6 ~ Fable, GPT 4.5 ~ Codex 5.astra
기술	C++20, DirectX 12, DXR, HLSL, ReSTIR PT, DLSS 4.5, Unity Exporter
핵심 성과	RTX 4080 기준 1080p 100 FPS, 1440p 50-60 FPS Real-time Path Tracing
수상	국내 게임학회 학술대회 우수상




제 2026-159호

우 수 발 표 상

성명(소속): 이윤상(한국공학대학교)
제목: 실시간 패스 트레이싱을 위한 객체-데이터 하이브리드 게임 클라이언트 프레임워크

위 사람은 2026년도 한국게임학회 춘계 학술 발표대회에서 상기 논문을 발표하여 본 학회의 심사를 거쳐 우수발표상 수상자로 선정되었기에 상장을 수여합니다.

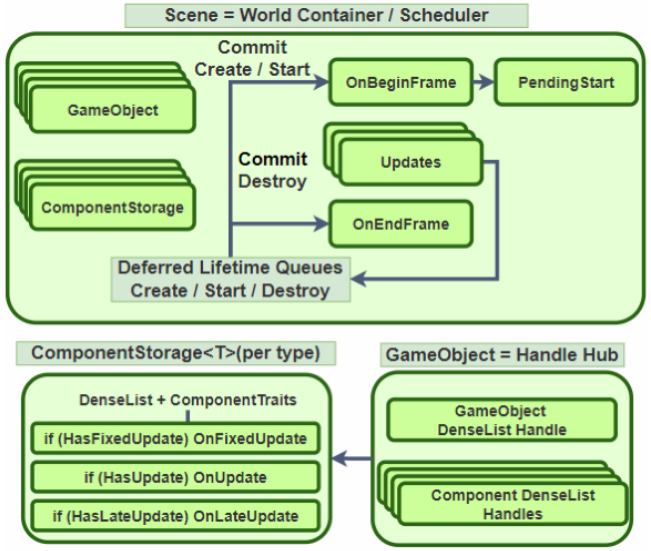
2026년 5월 30일

사단법인 한국게임학회
회 장 윤 태 복

프레임워크 오브젝트 정의: 조립과 데이터 저장

GameObject-Component 기반의 조립 방식을 유지하면서, 매 프레임 반복되는 타입별 컴포넌트 순회를 효율화해야 했습니다.

오브젝트는 핸들을 보관하고, 데이터는 타입별로 저장



논문 그림 2: Scene 기반 오브젝트-데이터 하이브리드 구조

DenseList – Handle코드

```cpp
// DenseList
// - 장기 식별자는 Handle만 사용.
// - pointer/reference/iterator/span은 컨테이너 변경(Emplace/Remove/Clear/Reserve 등)
//     이후 무효화될 수 있으므로 프레임/루프를 넘어 저장 금지.
// - 순회 중 임시 사용은 허용.

template <typename T>
struct DenseListHandle
{
    uint32_t id = 0;
    uint32_t generation = 0;

    DenseListHandle() = default;
    DenseListHandle(uint32_t _id, uint32_t _gen) : id(_id), generation(_gen) {}

    static DenseListHandle Invalid() { return DenseListHandle{0, 0}; }

    // 오래된 핸들인지 실제 저장소와 대조하는 검사는 DenseList::IsValid(handle)에서 수행
    bool IsValid() const { return generation != 0; }

    static DenseListHandle FromValue(uint64_t value) noexcept
    {
        return DenseListHandle{static_cast<uint32_t>(value & 0xFFFFFFFF), static_cast<uint32_t>(value >> 32)};
    }

    uint64_t GetValue() const noexcept { return (static_cast<uint64_t>(generation) << 32) | id; }

    auto operator<=>(const DenseListHandle&) const = default;
};
```

DenseList - Remove코드

```cpp
void Remove(Handle handle)
{
    if (!IsValid(handle))
    {
        return;
    }
    const uint32_t id = handle.id;
    const uint32_t index = m_idToIndex[id];
    const uint32_t lastIndex = static_cast<uint32_t>(m_data.size() - 1);
    const uint32_t lastId = m_indexToId[lastIndex];

    if (index != lastIndex)
    {
        m_data[index] = std::move(m_data[lastIndex]); // std::vector<T> m_data

        m_idToIndex[lastId] = index;
        m_indexToId[index] = lastId;
    }
    m_data.pop_back();
    m_indexToId.pop_back(); // std::vector<uint32_t> m_indexToId

    m_idToIndex[id] = kInvalidIndex; // std::vector<uint32_t> m_idToIndex
    BumpGeneration(m_generation[handle.id]); // std::vector<uint32_t> m_generation
    m_freeIds.push_back(handle.id); // std::vector<uint32_t> m_freeIds
}
```

ComponentStorage<T>가 DenseList<T>를 소유합니다.
같은 타입의 데이터를 순회하고, 지원하는 업데이트만 실행합니다

ComponentStorage.h - OnUpdate 코드

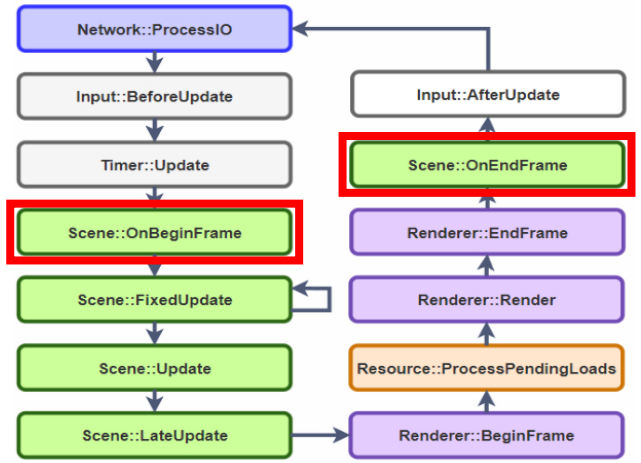
```cpp
void OnUpdate(float deltaTime) override
{
    if constexpr (ComponentTraits::HasUpdate<T>)
    {
        for (auto& component : m_data) // DenseList<T> m_data
        {
            if (component.IsEnabled() && component.IsOwnerActiveInHierarchy())
            {
                component.OnUpdate(deltaTime);
            }
        }
    }
}
```

논문 순회 측정: 4천만 회, ns/op. DenseList는 삽입·삭제 시 핸들·매핑 관리 비용을 부담합니다.
실제 핫루프 환경과 다른 별도 테스트 환경에서의 측정 값입니다.

DenseList	std::vector	std::vector<unique_ptr>	SparseSlotVector
1.84	1.78	4.77	5.59

프레임워크 오브젝트 라이프사이클: 생성, 삭제 요청의 지연 처리

ComponentStorage는 추가, 삭제 시 원소 위치가 바뀔 수 있습니다. 순회 중 변경 요청과 실제 컨테이너 변경을 분리해야 했습니다.



논문 그림 1: 생성·갱신·렌더링·삭제의 프레임 실행 순서

Scene.cpp - 실제 생성·삭제 반영 지점

```cpp
void Scene::OnBeginFrame()
{
    PixScopedCpuEvent event(L"Scene.OnBeginFrame");
    ProcessDeferredCreates();
    ProcessDeferredComponentCreates();
    ProcessPendingStarts();
}

void Scene::OnEndFrame()
{
    PixScopedCpuEvent event(L"Scene.OnEndFrame");
    ProcessDeferredComponentDestroys();
    ProcessDeferredDestroys();
}
```

생성·시작은 프레임 초반, 삭제는 프레임 종료에 반영

Scene이 요청을 지연 큐에 모아 정해진 단계에서 처리합니다.
핸들의 id/generation 검사로 삭제·재사용된 슬롯을 구분합니다.
삭제 지연처리는 DenseList::Remove가 호출되고 생성시에 처리가 필요하여 Reserve-FulFill 형태로 지연 생성처리 하였습니다.
Scene::ReserveGameObject만을 통해 오브젝트를 생성하게 설계했습니다.

Scene.cpp - 오브젝트 생성 함수

```cpp
GameObject::Handle Scene::ReserveGameObject(
    std::string name, std::optional<ServerID> serverID, std::function<void(GameObject*)> onCreated
)
{
    GameObject::Handle handle = m_gameObjects.ReserveHandle();

    if (serverID.has_value())
    {
        RegisterNetworkObject(serverID.value(), handle);
    }

    m_pendingCreates.push(CreateRequest{
        .name = std::move(name), .handle = handle, .serverID = serverID, .onCreated = std::move(onCreated)
    });

    return handle;
}
```

DenseList.h - 생성 예약 (Scene::ReserveGameObject 내 호출)

```cpp
[[nodiscard]] Handle ReserveHandle()
{
    uint32_t id{};

    if (!m_freeIds.empty())
    {
        id = m_freeIds.back();
        m_freeIds.pop_back();
    }
    else
    {
        assert(m_generation.size() < static_cast<size_type>
            (std::numeric_limits<uint32_t>::max()));
        id = static_cast<uint32_t>(m_generation.size());
        m_generation.push_back(m_baseGeneration);
        m_idToIndex.push_back(kInvalidIndex);
    }

    return Handle{id, m_generation[id]};
}
```

DenseList.h – 생성 반영 (Scene::ProcessDeferredCreates 내 호출)

```cpp
template <typename... Args>
    requires std::constructible_from<T, Args...>
void FulfillReservation(Handle handle, Args&&... args)
{
    if (!IsReserved(handle))
    {
        assert(false && "[DenseList] FulfillReservation - Invalid handle passed");
        return;
    }

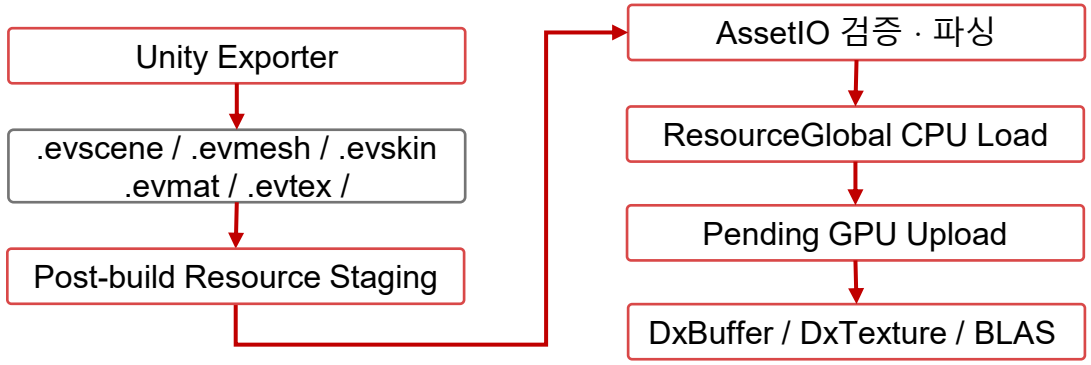
    if (m_idToIndex[handle.id] != kInvalidIndex)
    {
        assert(false && "[DenseList] FulfillReservation - Handle already fulfilled");
        return;
    }

    const uint32_t index = static_cast<uint32_t>(m_data.size());
    m_data.emplace_back(std::forward<Args>(args)...);
    m_indexToId.push_back(handle.id);
    m_idToIndex[handle.id] = index;
}
```

리소스 Export Tool & Other Tools

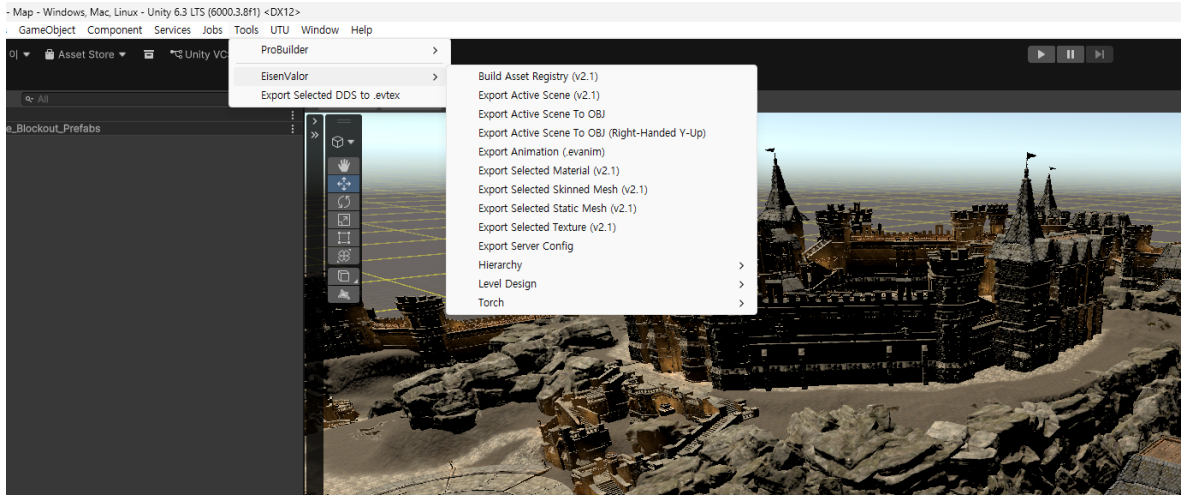
Unity Exporter

Unity 원본 데이터를 런타임 전용 바이너리 포맷으로 변환해, DCC 포맷 파싱과 게임 오브젝트 조립 책임을 분리했습니다.
AssetIO는 데이터 검증·파싱을, ResourceGlobal은 CPU 로드와 프레임 내 GPU 업로드를 담당합니다.
모든 리소스는 GUID 기반으로 관리합니다.



게임 오브젝트 조립

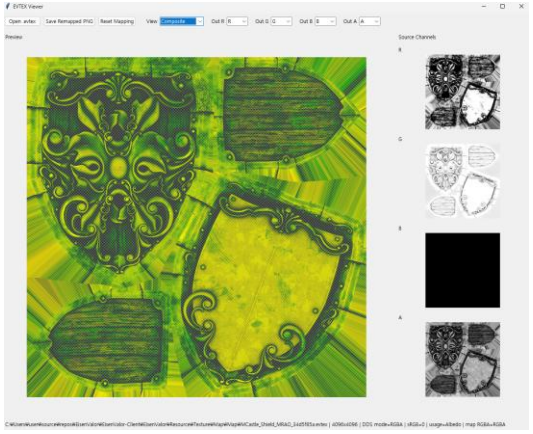
Scene 로더가 배치와 기본 컴포넌트를 복원합니다.
게임 전용 메타데이터는 등록된 decoder에서 해석합니다.



빌드 시 Resource 폴더를 실행 디렉터리에 배치

그 외의 툴

AI를 활용해 Python·HTML 기반으로 필요한 툴을 빠르게 제작해냈습니다



.evtex 채널 확인, 재매핑



입력 바인딩 시각화

프레임워크 리소스: 조회, 업로드, 해제 시점 관리

리소스 조회와 GPU 작업을 분리하고, GPU가 사용 중인 자원은 fence 완료 이후에 해제하도록 구성했습니다.

GUID 조회·캐시 → 메타데이터·요청 등록 → 프레임 내 업로드 → GPU 리소스 사용 → 해제 요청·fence 연결 → 완료 확인·실제 해제

01 캐시 재사용과 로드 예약

CPU에서 리소스를 얻는 시점

메시·텍스처는 메타데이터와 리소스 객체를 준비한 뒤 GPU 업로드 요청을 큐에 등록합니다.

02 현재 프레임에 업로드 기록

GPU 작업의 기록·실행

업로드 단계에서 데이터를 읽고 복사 명령을 기록합니다. 남은 업로드 용량이 부족하면 다음 프레임으로 미룹니다.

03 완료된 항목의 해제 실행

참조 해제와 GPU 완료는 별도

해제 요청에 연결된 fence와 큐 완료 값을 비교합니다. 완료된 항목만 콜백을 실행해 실제 자원을 정리합니다.

ResourceGlobal.h - 01 조회와 캐시 등록

```cpp
template <IsValidResource T>
std::shared_ptr<T> Load(const EvAsset::Guid& guid)
{
    if (auto cache = GetResource<T>(guid))
    {
        return cache;
    }

    auto it = m_guidToPath.find(guid);
    if (m_guidToPath.end() == it)
    {
        return nullptr;
    }

    std::shared_ptr<T> resource = LoadInternal<T>(it->second);
    if (resource)
    {
        m_resourceCache[guid] = resource;
        m_pathToGuid[it->second.wstring()] = guid;
        m_guidToLastWriteTime[guid] = std::filesystem::last_write_time(it->second);
    }

    return resource;
}
```

ResourceGlobal.cpp - 01 메시 업로드 요청

```cpp
template <>
std::shared_ptr<MeshResource>
ResourceGlobal::LoadInternal<MeshResource>(const std::filesystem::path& path)
{
    EvAsset::MeshData data;
    if (false == EvAsset::AssetLoader::Load(path, data))
    {
        DEBUG_LOG_FMT("[ResourceGlobal] Failed to load skinned mesh: {}\n", path.string());
        return nullptr;
    }
    auto res = std::make_shared<MeshResource>();
    res->SetGuid(data.assetGuid);
    res->SetName(data.name);
    res->SetMetadata(...);

    m_pendingLoads.push({res, data.assetGuid, path, MeshResource::StaticRuntimeTypeID()});

    return res;
}
```

ResourceGlobal.cpp - 02 예약된 업로드 처리

```cpp
void ResourceGlobal::ProcessPendingLoads()
{
    if (m_pendingLoads.empty())
    {
        return;
    }

    auto* frame = GLOBAL(DxRendererGlobal).GetCurrentFrame();
    if (!frame)
    {
        return;
    }
    auto* device = GLOBAL(DxDeviceGlobal).GetDevice();
    auto& context = *frame->GetMainContext();
    auto* cmdList = context.CommandList();
    auto* uploadHeap = frame->GetUploadHeap();

    // ... 업로드 예산 계산 및 용량 검사 함수

    while (!m_pendingLoads.empty())
    {
        LoadingTask task = m_pendingLoads.front();

        // ... 각 타입별 업로드 처리
    }
}
```

DxGarbageCollectorGlobal.cpp - 03 완료 확인 및 해제 실행

```cpp
// ProcessCompleted() 내부 발췌

const auto& e = q.front();

if (snapshotDone >= e.fenceHandle.value)
// 해당 GPU 큐의 완료 fence 값 >= 이 항목을 해제할 수 있는 fence 기준값
{
    auto fn =
        std::move(q.front().releaseCallback);
    auto debugName =
        std::move(q.front().debugName);
    q.pop_front();
    try
    {
        fn();
    }
    catch (const std::exception& ex)
    {
// ... 예외처리
    }
    ++m_totalProcessed;
}
```

프레임워크 렌더러: 데이터 준비 및 패스 간 전달

각 패스가 저장소와 수명을 관리하고, RenderContext는 현재 프레임의 데이터를 타입별로 공개·조회합니다.

Skinning → DXR / 후보 생성 → Temporal Reuse → Final Evaluation → DLSS → HDR Resolve → UI

현재 등록된 패스 흐름 - Temporal / DLSS는 논문 이후 추가된 패스

저장 정책을 소유 지점에서 결정

FrameBuffered
프레임 리소스별 저장소 – 인스턴스, 재질, 지오메트리

PingPongHistory
이전 읽기 / 현재 쓰기 - Temporal history

Persistent
명시적 무효화까지 유지 - 정적 씬 테이블

Transient
현재 프레임의 공개 데이터 - 후보, 최종 reservoir 뷰

RenderData는 타입이 있는 payload입니다.
저장 정책과 reset, swap, resize 결정은
이를 소유한 패스에 둡니다.

DxrRenderPass.h - 실제 저장소 선언

```
FrameBuffered<TlasRenderData, 3> m_tlas;
FrameBuffered<RaytracingOutputRenderData, 3> m_outputData;
Transient<RestirCandidateRenderData> m_restirCandidateData;

FrameBuffered<InstanceRenderData, 3> m_instanceData;
FrameBuffered<MaterialRenderData, 3> m_materialData;
FrameBuffered<GeoTableRenderData, 3> m_geoTableData;
FrameBuffered<RestirLightRenderData, 3> m_restirLightData;
```

DxrRenderPass.cpp - 데이터 셋팅

```
renderContext->Set(m_instanceData);
renderContext->Set(m_materialData);
renderContext->Set(m_geoTableData);
renderContext->Set(m_restirLightData);
renderContext->Set(m_tlas);
renderContext->Set(m_outputData);
renderContext->Set(m_restirCandidateData);
```

RestirTemporalReusePass.cpp - 데이터 타입별 조회

```
// ... Execute 내부 발췌
auto* cameraData = renderContext->Get<CameraRenderData>();
auto* candidateData = renderContext->Get<RestirCandidateRenderData>();
auto* tlasData = renderContext->Get<TlasRenderData>();
auto* instanceData = renderContext->Get<InstanceRenderData>();
auto* materialData = renderContext->Get<MaterialRenderData>();
auto* geoTableData = renderContext->Get<GeoTableRenderData>();
```

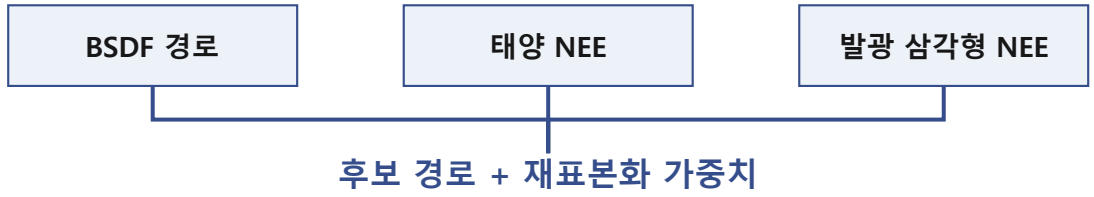
RestirTemporalReusePass.cpp - PingPongHistory swap 결정

```
// ... DispatchTemporalReuse 내부 발췌
m_temporalHistory.Swap();
m_historyValid = true;
return true;
```

각 패스 객체에 직접 접근하지 않고 RenderContext만 참조하여 필요한 데이터 타입을 조회하도록 구성하였습니다.

패스 트레이싱: ReSTIR PT path 후보 생성과 reservoir 선택

BSDF 경로와 태양·발광체 직접 조명 후보를 생성하고, 재표본화 가중치에 따라 reservoir에 저장할 후보를 선택하도록 구현했습니다.



01 후보를 공통 선택 함수에 전달

각 방식으로 생성된 후보의 경로 정보와 가중치를 동일한 reservoir 갱신 함수에 전달합니다.

02 누적 가중치에 따라 대표 경로 선택

새 후보의 가중치를 누적한 뒤, 전체에서 차지하는 비율을 교체 확률로 사용합니다. 이 확률에 따라 기존 경로를 유지하거나 새 후보로 교체합니다.

03 선택 경로와 가중치 정보 보존

선택 경로·누적 가중치·선택된 후보의 가중치를 저장하고, 생성된 reservoir를 Temporal에 전달합니다.

RestirReservoir.hlsli · 가중치 누적과 대표 경로 선택

```
void RestirUpdateReservoir(
    inout RestirReservoir reservoir,
    RestirPathSample candidate,
    float resamplingWeight,
    float randomValue)
{
    static const float maxFinite = 3.402823466e+38f;
    if (!(resamplingWeight > 0.0f && resamplingWeight < maxFinite))
    {
        return;
    }

    float updatedWeightSum = reservoir.resamplingWeightSum + resamplingWeight;
    if (!(updatedWeightSum > 0.0f && updatedWeightSum < maxFinite))
    {
        return;
    }
    reservoir.resamplingWeightSum = updatedWeightSum;

    float selectionProbability = resamplingWeight / max(reservoir.resamplingWeightSum, EPSILON);
    if (randomValue <= selectionProbability)
    {
        reservoir.sample = candidate;
        reservoir.selectedResamplingWeight = resamplingWeight;
        reservoir.flags = RESTIR_RESERVOIR_VALID;
    }
}
```

교체 확률 = 새 후보의 가중치 / 새 후보를 포함한 누적 가중치

ReSTIR PT: Temporal path reuse와 가중치 처리

이전 프레임의 path를 현재 표면에서 재평가하고, 현재, 이전 후보의 가중치를 계산해 reservoir를 갱신하도록 구현했습니다.

이전 픽셀 재투영 → 표면·history 검증 → Path reconnection → Multiple Importance Sampling 가중치 계산·재선택 → 최종 평가·history 저장

01 현재 후보의 MIS 가중치 계산

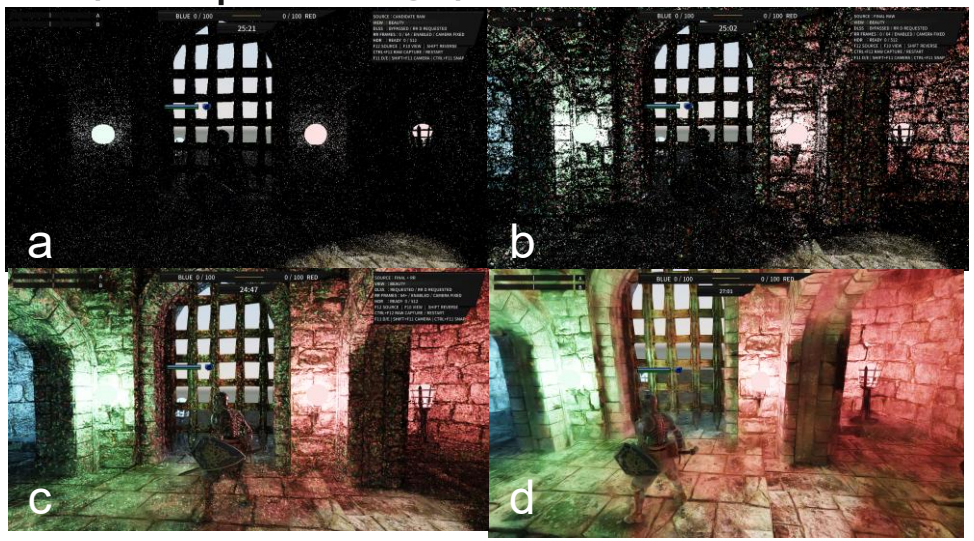
```
float currentSelfDensity = currentCandidate.weightTerms.x * currentM;
float previousDomainDensity = hasBackwardShift
        ? backwardTarget * backwardJacobian * cappedHistoryM
        : 0.0f;
float currentMisWeight = currentSelfDensity /
        max(currentSelfDensity + previousDomainDensity, EPSILON);
currentCandidate.weightTerms.z = currentMisWeight;
```

02 재연결된 이전 후보의 MIS 가중치 계산

```
float currentDomainDensity = forwardTarget * currentM;
float previousSelfDensity = previousTarget /
        max(forwardJacobian, EPSILON) * cappedHistoryM;
float previousMisWeight = previousSelfDensity /
        max(currentDomainDensity + previousSelfDensity, EPSILON);
reconnectedCandidate.weightTerms.z = previousMisWeight;
```

각 후보의 resampling된 가중치에 MIS 가중치를 곱한 뒤,
앞 페이지의 RestirUpdateReservoir()의 인자로 동일하게 outputReservoir를 갱신합니다.

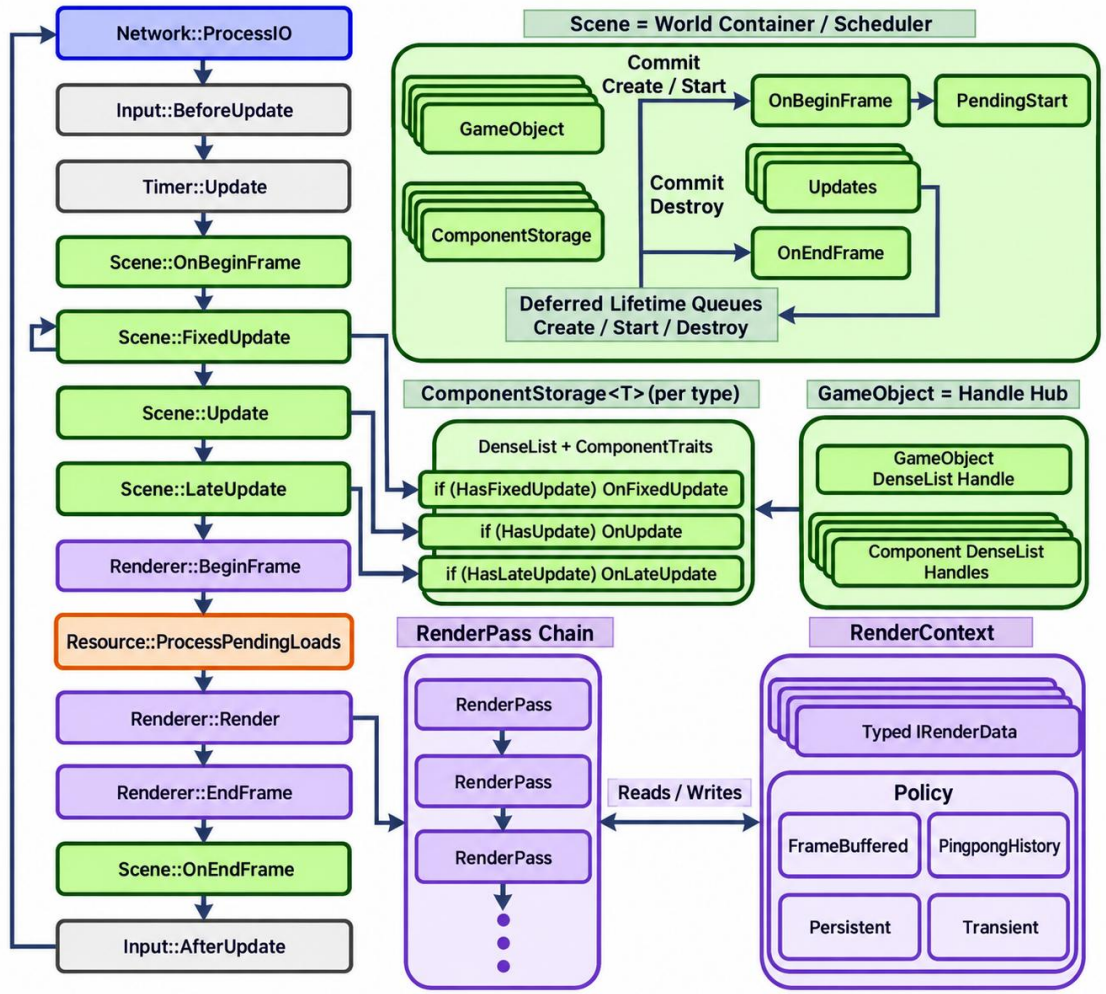
최신 실행 결과 - Temporal 조명 상태 확인



a: Temporal X / DLSS X, b: Temporal O / DLSS X, c: Temporal O / DLSS O, d: RR 초기 화면

수정 완료	RR 입력용 Skinned MV 가이드의 transform 오류
검증 결과	c와 d의 Temporal 경로 재연결·재표본화 가중치 정합성은 성립 이후 저자의 프로젝트로 테스트 결과, Spatial없이 Temporal만 사용하는 경우 동일한 문제 발견
추가조치	RR의 fresh 비율을 0%에서 70%로 변경하여

전체 프레임워크: 실행과 데이터의 연결



전체 구조: 프레임 실행·씬 생명주기·타입별 저장소·렌더 패스

앞에서 설명한 오브젝트, 리소스, 지연 처리, 렌더링을 한 프레임의 실행과 데이터 흐름으로 정리했습니다.

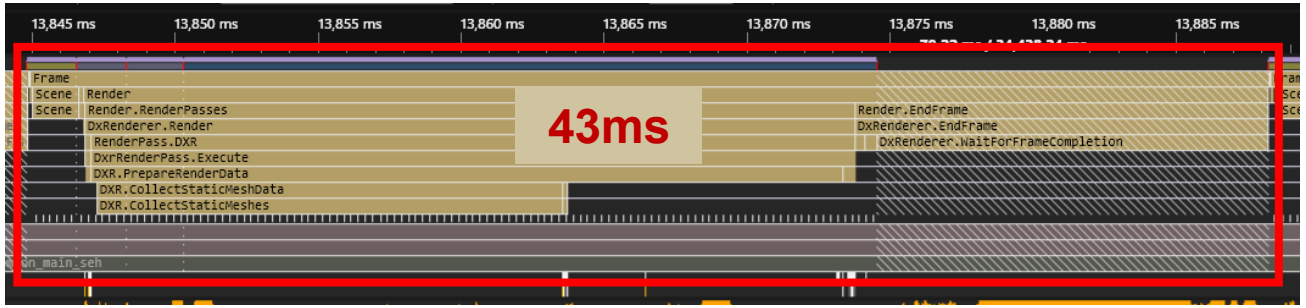
게임 전용 코드와 공통 런타임

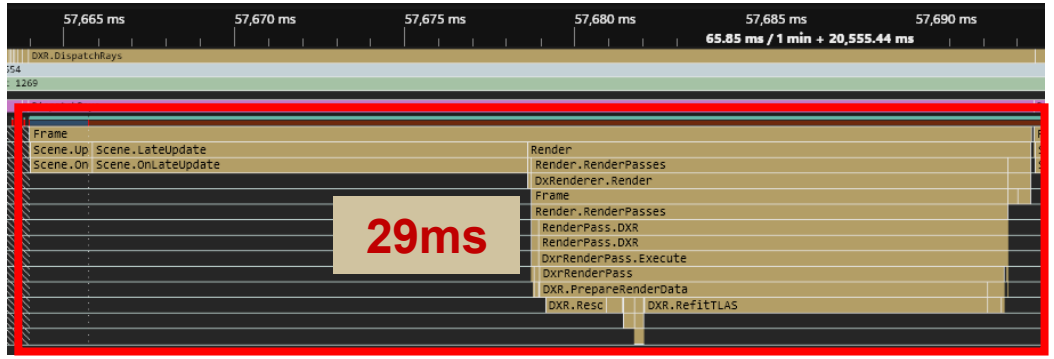
EisenValor 실제 게임으로 씬, 컴포넌트, 렌더 패스를 조립합니다.
ClientFramework는 공통 실행과 서비스를 제공합니다.

공통 런타임	역할
GameFramework	초기화, 프레임 실행 순서, 종료
SceneGlobal / Scene	활성 씬, 객체, 업데이트, 생명주기
Input / Timer / Network	입력, 시간 갱신, 네트워크 I/O
ResourceGlobal	리소스 조회, GPU 업로드 예약
DxRendererGlobal	프레임 준비, 패스 실행, GPU 제출

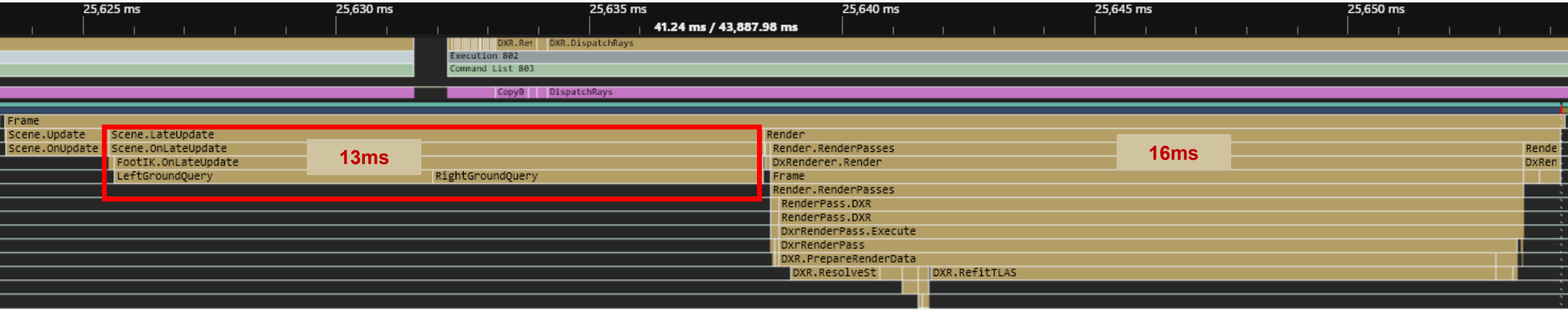
현재 코드의 소유 관계: Policy 저장소는 각 RenderPass가 소유하고,
RenderContext는 현재 RenderData를 타입별로 공개·조회합니다.

PIX를 활용한 프로파일링



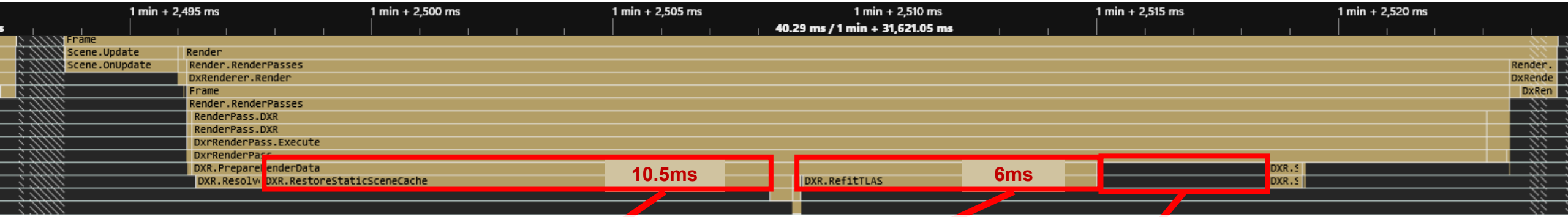


- 프레임 끝에 대기하는 부분이 있어 확인 결과, 필요없는 gpu대기를 넣었어서 제거하였습니다.
- StaticObject 수집이 매 프레임 반복되어 StaticObject는 초기에 1회만 수집하여 별도로 처리하였습니다.

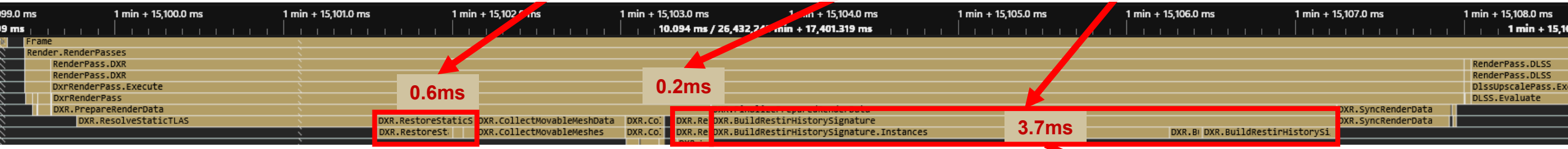


- 그러고 나서도 프레임이 낮았고 LateUpdate에 마커를 추가해본 결과,
  FootIK 지면 질의가 비효율적임을 확인하여 해당 코드 담당인 팀원분께 전달하였습니다.

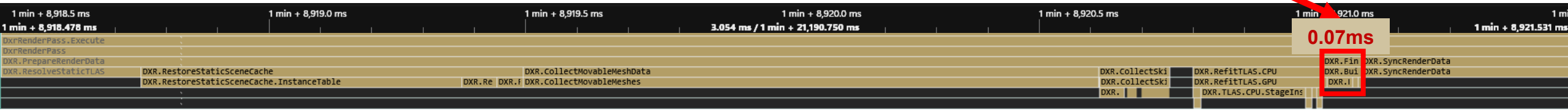
PIX를 활용한 프로파일링



- 이후 IK관련은 수정되었고 그 다음 문제는 MotionVector 생성관련 수정 후의 Instance 순회 과정에서 Static까지 모두 순회해버린 것에서 발생했습니다. 추가적으로 TLAS에서도 해당 부분이 있음을 발견하여 같이 수정하였습니다.



- 추가적으로 Temporal History 호환성 검증용 해시에서도 Static까지 모두 순회하던 부분을 수정하였습니다.



- 해당 시점에서 DxrRenderPass 실행 속도가 최초 24.7ms에서 4.5ms로 5.5배 빨라졌고 CPU bound에서 GPU bound로 이동하였습니다.

콘솔 기반 런타임 진단

해당 프로젝트에서 PIX의 역할은 단일 프레임의 CPU·GPU 실행시간과 큐 병목 분석이였습니다.
하지만 이후 DXGI_ERROR_DEVICE_HUNG에 대한 디버깅을 하면서 정보가 부족함을 실감했습니다. 따라서 디버깅을 위해 다음과 같은 발전 과정이 있었습니다. AI(codex 5.6 sol high-xhigh)를 활용하여 해당 로그 폴더를 읽도록 skills를 운용하였습니다.

Debug Layer · GPU-Based Validation

잘못된 리소스 상태·Descriptor·GPU 접근 검증으로 시작했습니다

InfoQueue

Warning/Error/Corruption 분류·출력으로 디버깅을 세분화하여 로그 내 서치를 더 간편하게 바꿨습니다.

그러나 지속적인 GPU TDR(Timeout Detection and Recovery) 문제 발생

DRED (Device Removed Extended Data)

GPU가 멈추기 전 마지막 명령어 추적,
GPU Fage Fault 시 연관 API 리소스 이름과 종류
추적의 기능을 수행합니다

```
[DxrRenderPass] TLAS Refit requested
[DxTLAS] RefitRTAS: ...
[DRED] Device removed ... Reason: 0x887A0006
[DRED] op[22] = 15  <-- last completed
[DRED] op[23] = 31  <-- potential fault
[DRED] Faulting VA: 0x0000000000000000
```

- 0x887A0006 = DXGI_ERROR_DEVICE_HUNG
- DRED Breadcrumb의 op = 31 = BUILD_RAYTRACING_ACCELERATION_STRUCTURE
- 바로 직전 엔진 로그가 TLAS Refit requested와 DxTLAS RefitRTAS

이렇게 예시처럼 어떤 시점에 어느 명령이 무슨 이유로 발생한 TDR임을 확인할 수 있게 되었음

단순한 DXGI_ERROR_DEVICE_HUNG를
어느 CommandList의 어떤 명령과 리소스에서 멈췄는가로 구체화

프로파일과 로그를 동시에 AI로 분석

후보 생성 경로별 측정			
모드	Candidate GPU 타임	대비 증가한 시간	전체 GPU Frame시간
PATH	3.995ms	-	12.854ms
PATH + SUN	4.912ms	+0.917ms	13.955ms
FULL	12.530ms	**+7.618ms**	21.300ms

GPU bound로 이동한 시점에서 하나의 DispatchRays 내부 후보 생성 경로의 레이 후보를 생성한 부분을 테스트하기 위해
- **PATH**: 기본 BSDF 경로 후보
- **PATH+SUN**: PATH + 태양 NEE 후보
- **FULL**: PATH+SUN + emissive NEE 후보

세가지로 단계적 활성화하는 모드들로 진행하였습니다

Emissive NEE 추가 시 비용이 크게 증가해, 내부 작업을 단계적으로 활성화하고 모드 간 GPU 시간 차이로 병목 구간을 좁혔습니다.

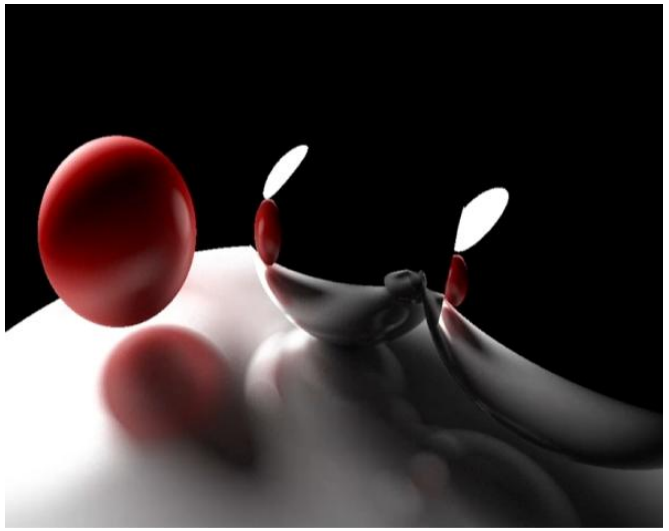
- **PATH + SUN**: emissive 후보 생성이 없는 상태
- **RETRACE SURFACE**: 위 작업
  + 동일 primary ray 재추적 및 shading point의 표면/재질 재구성
- **SAMPLE NO VISIBILITY**: 위 작업 + light 선택, 삼각형 샘플링, BSDF/PDF 계산
- **FULL**: 위 작업 + shadow RayQuery

코드에서 618개 emissive 엔트리를 순차 탐색하는 루프를 확인했고, 이를 우선 최적화 대상으로 선정했습니다. 픽셀별 탐색 종료 위치 차이에 따른 wave divergence도 비용 증가 요인으로 추정했습니다.

Emissive 경로에 대한 상세 모드 측정		
모드	Candidate GPU 타임	대비 증가한 시간
PATH + SUN	3.790ms	-
RETRACE SURFACE	3.952ms	+0.162ms
SAMPLE NO VISIBILITY	8.577ms	**+4.625ms**
FULL	8.873ms	+0.296ms

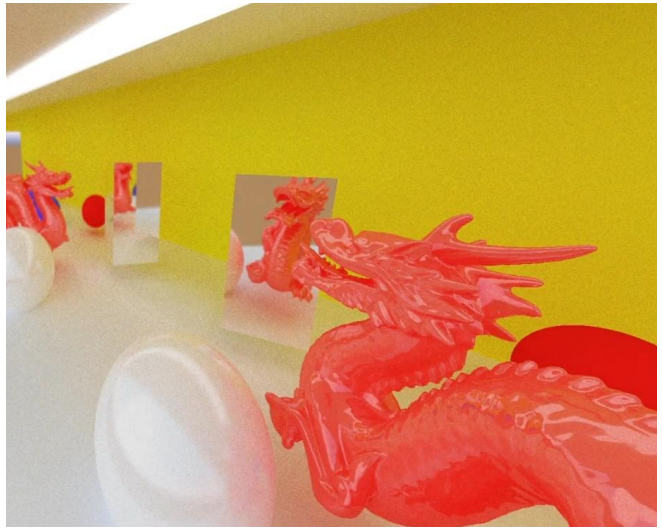
실행 조건 확인에는 emissive물체 개수나 animated BLAS 수의 로그를 활용했으며, 병목 식별의 핵심 근거는 각 모드 간 GPU 시간 차이였습니다.
CPU에서 **emissive weight의 누적합(CDF)을 구성**하고, GPU의 **선형 탐색을 이진 탐색으로 교체**했습니다. 618개 엔트리를 **최대 618회 검사하던 과정을 최대 10회** 비교로 줄였습니다.

측정 항목	CDF 적용 전	적용 후	감소율
FULL Candidate GPU 시간	8.873ms	4.622ms	**47.9%**
전체 GPU Frame	15.587ms	12.457ms	**20.1%**









RayTracingSimulator

C++ Framework | BVH | OpenGL | GLSL Compute

- Compute Shader에서 pixel별 ray를 생성하고 bounce count와 ray per pixel을 조절했습니다.
- SSBO로 sphere/material data를 전달하고 frame accumulation으로 노이즈를 줄였습니다.
- BVH 탐색과 ImGui/Gizmo를 붙이며 이후 EisenValor DXR 구조로 확장할 지식 기반을 만들었습니다.







METALSLUG Clone

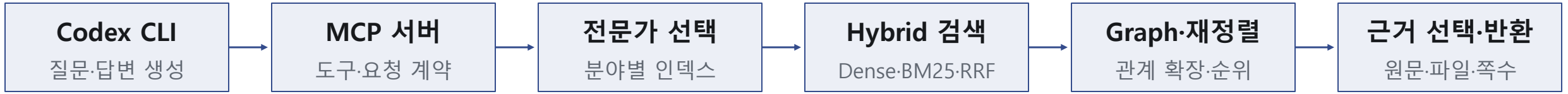
WinAPI 기반 2D 액션 게임. 자체 OOP클라이언트 프레임워크 구조, 상하체 분리된 애니메이션, 충돌, 몬스터 패턴을 구현했습니다.

Animation	플레이어 상/하체 상태와 공격, 점프, 앉기 애니메이션
Collision	Box/Circle collider와 enter/stay/exit 이벤트
Boss Pattern	보스 이동, 투사체, 이펙트, 사운드 스크립트
Resource	BMP/PNG, FMOD sound, text 기반 stage data

개인 문서 검색을 위한 RAG 확장

수업 RAG를 바탕으로 개인용 검색 도구를 직접 기획·설계하고, AI로 구현 코드를 생성해 빠르게 개발했습니다.

질의 처리 흐름과 모듈 경계



01 문제 상황

노트·API 문서·논문이 늘어나면서
구현 중 필요한 내용을 다시 찾는 일이
반복됐습니다. 또한 한·영 표현이 달라도
상황에 맞는 근거를 가져오고자 했습니다.

검색이 필요한 실제 구현 상황 중 하나

"동적 장면에서 Temporal reuse를
unbiased하게 유지하려면?"

한국어 구현 질문으로 영어 ReSTIR PT 논문 검색

02 해결방안

의미와 키워드 검색을 결합

기본적인 Dense와 BM25로 구현 개념과 함수명·전문 용어를
함께 검색하고, 후보를 합쳐 다시 정렬했습니다.

문서 관계를 Graph로 연결

인접 청크·공통 용어·코드 관계를 따라 후보를 추가해,
초기 검색에서 놓친 근거를 보완하도록 했습니다.

한·영 표현 차이에 대응

다국어 임베딩(BGE-M3)과 한국어 형태소 분석(Nori)을
적용하고, 한·영 질의를 바꿔 검색 결과를 점검했습니다.

EisenValor 자료 적용과 실제 CLI 질의

프로젝트 자료를 추가하고, 출처·원문을 반환하는 MCP를 실제 질의로 확인했습니다.

03 해결과정

1. EisenValor 자료로 확장

설계·계약 문서 11개 + 렌더링 소스 70개
관련 ReSTIR 논문 4편
프로젝트와 논문을 분야별로 검색합니다.

85개 자료 · 1,202청크

2. 검색 결과를 실제로 확인

한·영 질문 8건 모두 정상 응답
현재 진행 결정과 HLSL 원문 회수
과거 기록은 시점·출처 확인 필요

3. 응답 형식 개선

원문·출처 유지, 중복 본문·진단 정보 축소

기존 full	compact
69,188	**7,895**

Spatial 6질의 MCP 응답 토큰 88.6% 축소

o200k_base · CLI 전체 사용량·청구 비용 제외

실제 초기버전 Codex CLI 질의와 답변

질문 요지: 동적 장면의 Temporal reuse를 unbiased하게 유지하는 조건

① 실제 MCP 후속 검색 · 경로 변환과 가중치 보정

```
• Called
  └ llamaindex-rag.query_expert({"expert":"graphics_papers","question":"\"change of variables\" \"Jacobian\" shift
      mapping","top_k":5})
```

호출 질의: change of variables / Jacobian / shift mapping

② 같은 실행의 최종 답변 · 원문과 페이지 인용

```
• query_expert(expert="graphics_papers")를 실제 호출한 결과, 논문은 비편향 temporal reuse에 적절한 MIS 가중치가 필요하며
generalized Talbot 또는 pairwise MIS를 예로 듭니다(ReSTIR_PT.pdf, 23쪽).
프레임 간 경로 이동에는 정의된 영역에서 전단사인 temporal shift mapping이 필요하고, 성공한 이동은 역변환이 가능해야 하
므로 충돌하는 모션 벡터도 이 조건을 보존하도록 처리해야 합니다(ReSTIR_PT.pdf, 12·23쪽).
변수변환에는 야코비안 행렬식의 절댓값을 반영하고, 실현 가능한 각 경로에 대한 contribution MIS 가중치의 합이 1이 되도록
보정해야 합니다(ReSTIR_PT.pdf, 6–7쪽, 식 13·17).

재사용 후보에는 비편향 contribution weight가 따라야 하지만, 후보들의 독립성 자체는 필수 조건이 아니며 GRIS는 상관된 입
력도 허용합니다(ReSTIR_PT.pdf, 5쪽, §4.1–4.2).
또한 변환된 후보들의 지지집합이 적분할 함수 f의 지지집합을 덮어야 하며, 논문은 이를 충족하는 방법으로 현재 적분 영역에
서 직접 생성한 후보와 항등 매핑을 포함하는 구성을 제시합니다(ReSTIR_PT.pdf, 7쪽).
동적 장면에서 특히 중요한 조건은 현재·이전 프레임 양쪽에서 경로를 평가하는 것으로, 논문은 temporal MIS에서 가시성을 생
략하는 방법을 편향된 근사라고 명시합니다(ReSTIR_PT.pdf, 23쪽).

근거 부족: 이전 프레임 상태의 구체적인 저장·복원 방법이나 임의의 토폴로지 변경에 대응하는 매핑 구현은 이번에 반환된 원
문만으로 확인되지 않았습니다.
```

2026-10-29 보관 화면 · Astra(max) · 단일 논문·18회 호출 사례

직접 읽기 대비 토큰·컨텍스트 사용량

두 모델 모두 핵심 조건 4개를 근거와 함께 답했고, RAG의 최대 요청 입력이 33.1–36.2% 줄었습니다.

04 결과 - 키워드 직접 검색·페이지 읽기와 RAG 비교

동일 질문·논문 4편(72쪽), 추론 max, 각 방식 최대 3회 조회

네 실행 모두 핵심 근거 4개 확인

GPT-6-Astra와 GPT-6.1-Sol을 방법별 각 1회 실행 · 2026-11-10 실측

측정 항목 (직접 읽기 → RAG)	GPT-6-Astra	GPT-6.1-Sol
최대 요청 입력 컨텍스트 한 요청의 최대 입력량, 캐시 포함	55,209 → 35,218 **36.2% 감소**	56,505 → 37,790 **33.1% 감소**
누적 입력+출력 모든 모델 요청의 합계, 캐시 포함	162,394 → 140,693 **13.4% 감소**	166,354 → 151,898 **8.7% 감소**
캐시 제외 입력 CLI가 캐시로 분류하지 않은 입력	53,180 → 35,082 **34.0% 감소**	54,390 → 37,636 **30.8% 감소**
모델에 전달된 도구 텍스트 출력 잘림 이후 실제 전달 내용	30,181 → 11,985 **60.3% 감소**	32,484 → 12,203 **62.4% 감소**

입력·출력: CLI 실제 usage / 도구 텍스트: o200k_base 비교 계수 / 네 세션의 자동 컨텍스트 압축: 0회

단일 질의의 실행 비교입니다. 일반 절감률·청구 비용·Graph 단독 기여도는 검증하지 않았습니다.

3DGS에 물리 정보를 부여하는 연구

3D Gaussian Splatting(3DGS)의 외형에 충돌체와 물성 정보를 결합해,
Unity에서 움직이는 강체 에셋으로 만드는 것을 목표로 했습니다.

담당: 프로젝트 기획·설계, 모델 제작·학습·실험 | 상태: 모듈별 검증 후 중단

프로젝트 입력과 목표 출력

입력 / 3DGS .ply

가우시안의 위치·크기·색 등으로
물체의 외형을 렌더링하는 데이터

목표 출력 / Unity 강체 에셋

기존 3DGS 외형 + 충돌체
+ 질량·마찰 등 물성 메타데이터

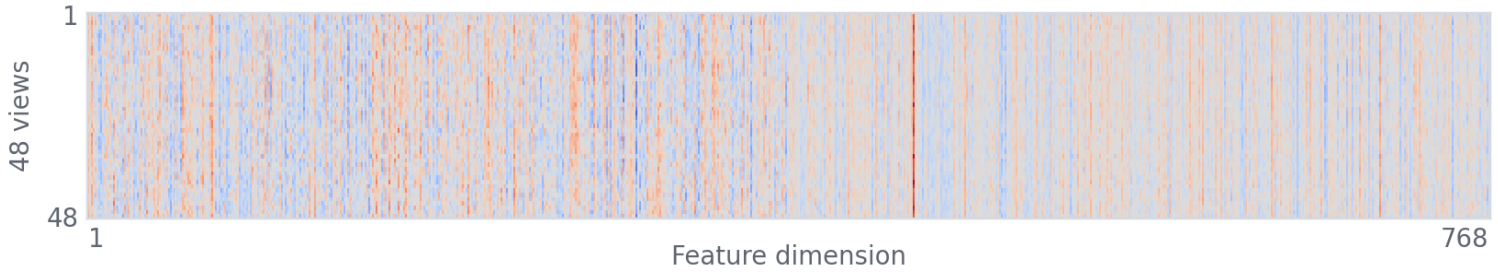
본인 모듈 / 물성 탐색용 사전정보

물체 특징 → 유사 사례 검색(LUT)
→ 밀도·마찰·반발계수의 초기 탐색 범위

최종 물성 보정과 에셋 통합은 미완료

모델 입력과 출력 · 동일 물체 예시

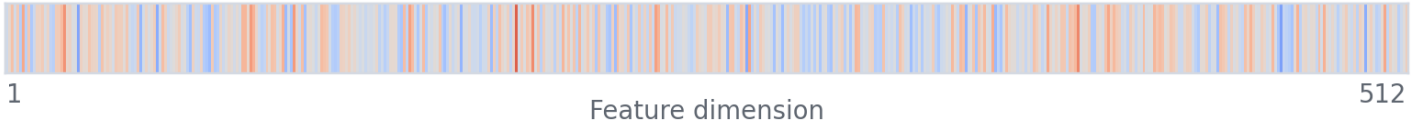
입력: 고정 DINOv2가 3DGS 48뷰에서 추출한 특징 [48 × 768]



각 행은 한 시점, 각 열은 특징 차원입니다.

학습한 Projection → 시점 평균 → 잔차 Fusion

출력: 이 물체를 대표하는 512차원 특징 z_obj [512]



같은 물체의 저장 특징으로 재계산 · 빨강: 양수 / 파랑: 음수 · 물성값이 아닌 특징값

특징 학습과 물성 예측을 나누어 검증

서로 다른 데이터셋의 두 실험입니다. 각 결과가 뒷받침하는 범위를 구분했습니다.

01 시점이 달라도 같은 물체를 찾는가

ShapeSplats 3,000개 · 물체 단위 분할
학습 2,100 / 검증 600 / 테스트 300

특징 표현	검색 Top-1 ↑
원본 DINO 특징 (768D)	96.00%
학습 특징 (768D projection)	98.49%
학습 특징 (512D + fusion)	**98.59%**

동일 물체 검색 성능 개선을 확인했습니다.

4~16뷰 부분집합 두 개를 매칭 · 테스트 300개 × 32회
부분집합 간 시점 중복 허용 · 물성 정확도 평가는 아님

02 시각 특징이 물성 prior에 도움이 되는가

GSO 300개 · 3회 분할 실험
재질 규칙으로 만든 약한 라벨 · 실측 물성 정답 없음

구간 중심값의 평균절대오차(MAE) · 낮을수록 좋음

물성	DINO + 회귀	카테고리 평균
밀도 (kg/m³)	613.68	614.05
마찰계수	0.0761	0.0628
반발계수	0.0270	0.0264

**카테고리 기준 대비 밀도는 비슷했고,
마찰·반발계수는 더 큰 오차를 보였습니다.**

두 표는 동일 입력의 전후 결과가 아닌, 독립적인 평가입니다.

설계 변경 | 직접 예측에서 LUT 기반 초기 탐색 범위 제안으로 조정

시뮬레이션 보정·메모리 갱신·최종 강체 에셋까지 연결한 검증은 완료하지 못했습니다.

Real-time Graphics & Engine

Graphics · Engine · AI for Graphics

GitHub	github.com/yunsang1ee 
Youtube	youtube.com/@profit_award
Main Focus	**Real-time Path Tracing for Game Clients**